

# Chuyên đề Quy Hoạch Động nâng cao

Bitmask DP

## Quy hoạch động trên bit (Bitmask DP)

- **Quy hoạch động:** Một phương pháp giải các bài toán truy hồi có thể sử dụng đệ quy theo hướng *khử đệ quy* nhằm giảm bớt việc tính toán trùng lặp trong đệ quy.
- **Quy hoạch động trên bit (bitmask DP):** Áp dụng phương pháp quy hoạch động mà các trạng thái được lưu trữ dưới dạng bitmask, các trạng thái này được tính toán thông qua các phép toán trên bit (bitwise operation).

# Các phép toán trên bit (bitwise operation)

- Gồm:
  - Phép AND: &
  - Phép OR: |
  - Phép XOR: ^
  - Phép NOT: ~

# Các phép toán trên bit (bitwise operation)

- Gồm:
  - Phép AND: &
  - Phép OR: |
  - Phép XOR: ^
  - Phép NOT: ~

| X | Y | X&Y | X Y | X^Y | ~(X) |
|---|---|-----|-----|-----|------|
| 0 | 0 | 0   | 0   | 0   | 1    |
| 0 | 1 | 0   | 1   | 1   | 1    |
| 1 | 0 | 0   | 1   | 1   | 0    |
| 1 | 1 | 1   | 1   | 0   | 0    |

# Các phép toán trên bit (bitwise operation)

- Gồm:
  - Phép AND: &
  - Phép OR: |
  - Phép XOR: ^
  - Phép NOT: ~
- Ví dụ, hãy tính kết quả của các phép tính sau:
  - $1011 \& 1101$
  - $1011 | 1101$
  - $1011 \wedge 1101$
  - $\sim 1011 | 1101$

# Các phép toán trên bit (bitwise operation)

- Gồm:
  - Phép AND: &
  - Phép OR: |
  - Phép XOR: ^
  - Phép NOT: ~
- Ví dụ, hãy tính kết quả của các phép tính sau:
  - $1011 \& 1101 = 1001$
  - $1011 | 1101 = 1111$
  - $1011 \wedge 1101 = 0110$
  - $\sim 1011 | 1101 = 0100 | 1101 = 1101$

# Các phép toán trên bit (bitwise operation)

- Gồm:
  - Phép AND: &
  - Phép OR: |
  - Phép XOR: ^
  - Phép NOT: ~
- Ví dụ, hãy tính kết quả của các phép tính sau:
  - $11 \& 13$
  - $11 | 13$
  - $11 \wedge 13$
  - $\sim 11 | 13$

# Các phép toán trên bit (bitwise operation)

- Gồm:
  - Phép AND:  $\&$
  - Phép OR:  $|$
  - Phép XOR:  $\wedge$
  - Phép NOT:  $\sim$
- Ví dụ, hãy tính kết quả của các phép tính sau:
  - $11 \& 13 = 9$
  - $11 | 13 = 15$
  - $11 \wedge 13 = 6$
  - $\sim 11 | 13 = 4 | 13 = 13$



# Các phép toán trên bit (bitwise operation)

- Gồm:
  - Phép AND: &
  - Phép OR: |
  - Phép XOR: ^
  - Phép NOT: ~
  - **Phép shift left (dịch bit qua trái): <<**
  - **Phép shift right (dịch bit qua phải): >>**

# Các phép toán trên bit (bitwise operation)

- Gồm:
  - Phép AND: &
  - Phép OR: |
  - Phép XOR: ^
  - Phép NOT: ~
  - **Phép shift left (dịch bit qua trái): <<**
  - **Phép shift right (dịch bit qua phải): >>**
- Ví dụ:
  - $1001 \ll 1 = 10010$
  - $1001 \ll 2 = 100100$

# Các phép toán trên bit (bitwise operation)

- Gồm:
  - Phép AND: &
  - Phép OR: |
  - Phép XOR: ^
  - Phép NOT: ~
  - **Phép shift left (dịch bit qua trái): <<**
  - **Phép shift right (dịch bit qua phải): >>**
- Ví dụ:
  - $1001 \ll 1 = 10010$       dec:  $9 \ll 1 = 18$
  - $1001 \ll 2 = 100100$       dec:  $9 \ll 2 = 36$

# Các phép toán trên bit (bitwise operation)

- Gồm:

- Phép AND: &
- Phép OR: |
- Phép XOR: ^
- Phép NOT: ~
- **Phép shift left (dịch bit qua trái): <<**
- **Phép shift right (dịch bit qua phải): >>**

- Ví dụ:

- $1001 \gg 1 = 100$                       dec:  $9 \gg 1 = 4$
- $1001 \gg 2 = 10$                       dec:  $9 \gg 2 = 2$

# Các phép toán trên bit (bitwise operation)

- Áp dụng:
  - Cho số nguyên  $x$ , hãy chỉnh bit thứ  $k$  của  $x$  thành giá trị 1 (không cần quan tâm trước đó bit thứ  $k$  của  $x$  là bao nhiêu).

# Các phép toán trên bit (bitwise operation)

- Áp dụng:
  - Cho số nguyên  $x$ , hãy chỉnh bit thứ  $k$  của  $x$  thành giá trị 1 (không cần quan tâm trước đó bit thứ  $k$  của  $x$  là bao nhiêu).
  - Kiểm tra xem bit thứ  $k$  của số nguyên  $x$  là 1 hay 0.

# Các phép toán trên bit (bitwise operation)

- Áp dụng:
  - Cho số nguyên  $x$ , hãy chỉnh bit thứ  $k$  của  $x$  thành giá trị 1 (không cần quan tâm trước đó bit thứ  $k$  của  $x$  là bao nhiêu).
  - Kiểm tra xem bit thứ  $k$  của số nguyên  $x$  là 1 hay 0.
  - Tính số lượng tập con của một tập hợp gồm  $n$  số nguyên phân biệt.

# Các phép toán trên bit (bitwise operation)

- Áp dụng:
  - Viết hàm tính tổng 2 số nguyên a và b mà chỉ dùng các phép toán trên bit.



# Các phép toán trên bit (bitwise operation)

- Áp dụng:
  - Viết hàm tính tổng 2 số nguyên a và b mà chỉ dùng các phép toán trên bit.
    - Gợi ý 1: phép XOR là phép cộng không nhớ.

# Các phép toán trên bit (bitwise operation)

- Áp dụng:
  - Viết hàm tính tổng 2 số nguyên a và b mà chỉ dùng các phép toán trên bit.
    - Gợi ý 1: phép XOR là phép cộng không nhớ.
    - Gợi ý 2: phép AND dùng để lấy ra vị trí có nhớ.

# Các phép toán trên bit (bitwise operation)

- Áp dụng:
  - Viết hàm tính tổng 2 số nguyên a và b mà chỉ dùng các phép toán trên bit.
    - Gợi ý 1: phép XOR là phép cộng không nhớ.
    - Gợi ý 2: phép AND dùng để lấy ra vị trí có nhớ.

=> Ý tưởng:

1. Lấy ra các vị trí cộng vào có nhớ ( $a \& b$ )
2. Cộng không nhớ ( $a \wedge b$ )
3. Dời các vị trí có nhớ qua trái (nhỏ  $\ll 1$ )
4. Thực hiện việc cộng nhớ vào kết quả ở bước 2
5. Lặp lại việc cộng nhớ vào cho đến khi nhớ bằng 0

# Các phép toán trên bit (bitwise operation)

- Áp dụng:
  - Viết hàm tính tổng 2 số nguyên a và b mà chỉ dùng các phép toán trên bit.

```
int tong(int a, int b) {  
    while (b > 0) {  
        int nho = a & b;  
        a ^= b;  
        b = nho << 1;  
    }  
    return a;  
}
```

# Các phép toán trên bit (bitwise operation)

- Áp dụng:
  - Viết hàm tính tổng 2 số nguyên a và b mà chỉ dùng các phép toán trên bit.
  - Viết hàm tính tích 2 số nguyên a và b mà chỉ dùng các phép toán trên bit.

# Các phép toán trên bit (bitwise operation)

- Áp dụng:
  - Viết hàm tính tích 2 số nguyên a và b mà chỉ dùng các phép toán trên bit.
- Gợi ý:
  - Gọi  $b = 2^{b_1} + 2^{b_2} + \dots + 2^{b_n}$
  - $a * b = a * (2^{b_1} + 2^{b_2} + \dots + 2^{b_n})$
  - $a * b = a * 2^{b_1} + a * 2^{b_2} + \dots + a * 2^{b_n}$
  - $a * b = \sum(a \ll b_i)$

# Các phép toán trên bit (bitwise operation)

- Áp dụng:
  - Viết hàm tính tích 2 số nguyên a và b mà chỉ dùng các phép toán trên bit.

```
int tich(int a, int b) {  
    int ans = 0;  
    while (b > 0) {  
        if ((b & 1) == 1) { // kiểm tra bit tận cùng bên phải của b có = 1 không  
            ans = tong(ans, a); // sử dụng hàm tổng của bài trước  
        }  
        a <<= 1;  
        b >>= 1;  
    }  
    return ans;  
}
```

## Quy hoạch động trạng thái với bitmask

- Bài toán mẫu:

Có  $n$  thành phố và  $m$  tuyến đường nối các thành phố với nhau. Các đường đi là một chiều. Bạn muốn đi du lịch qua tất cả các thành phố, mỗi thành phố đúng một lần. Bạn bắt đầu từ thành phố 0 đến thành phố  $n-1$ . Hãy đếm số cách di chuyển khác nhau. Kết quả chia lấy dư cho  $10^9+7$ .

Giới hạn:  $2 \leq n \leq 20$

<https://cses.fi/problemset/task/1690>



## Quy hoạch động trạng thái với bitmask

Nhận xét:

- Bài toán đếm  $\Rightarrow$  quy hoạch động.

## Quy hoạch động trạng thái với bitmask

Nhận xét:

- Bài toán đếm  $\Rightarrow$  quy hoạch động.
- Gọi  $F[i][S]$ : lưu số lượng cách di chuyển khác nhau từ thành phố 0 đến thành phố  $i$ , trong đó  $S$  là bitmask lưu tập hợp những thành phố đã thăm.

## Quy hoạch động trạng thái với bitmask

Nhận xét:

- Bài toán đếm  $\Rightarrow$  quy hoạch động.
- Gọi  $F[i][S]$ : lưu số lượng cách di chuyển khác nhau từ thành phố 0 đến thành phố  $i$ , trong đó  $S$  là bitmask lưu tập hợp những thành phố đã thăm.
- $F[i][S] = \sum F[j][S']$ 
  - với  $j$  kề với  $i$  và  $j$  thuộc  $S$  (bit  $j$  trong  $S = 1$ )
  - $S'$  là tập  $S$  nhưng không chứa  $i$ .

## Quy hoạch động trạng thái với bitmask

Nhận xét:

- Bài toán đếm  $\Rightarrow$  quy hoạch động.
- Gọi  $F[i][S]$ : lưu số lượng cách di chuyển khác nhau từ thành phố 0 đến thành phố  $i$ , trong đó  $S$  là bitmask lưu tập hợp những thành phố đã thăm.
- $F[i][S] = \sum F[j][S']$ 
  - với  $j$  kề với  $i$  và  $j$  thuộc  $S$  (bit  $j$  trong  $S = 1$ )
  - $S'$  là tập  $S$  nhưng không chứa  $i$ .
- Khởi tạo:  $F[0][1] = 1$

# Quy hoạch động trạng thái với bitmask

Nhận xét:

- Bài toán đếm => quy hoạch động.
- Gọi  $F[i][S]$ : lưu số lượng cách di chuyển khác nhau từ thành phố 0 đến thành phố  $i$ , trong đó  $S$  là bitmask lưu tập hợp những thành phố đã thăm.
- $F[i][S] = \sum F[j][S']$ 
  - với  $j$  kề với  $i$  và  $j$  thuộc  $S$  (bit  $j$  trong  $S = 1$ )
  - $S'$  là tập  $S$  nhưng không chứa  $i$ .
- Khởi tạo:  $F[0][1] = 1$
- Lưu ý: để kiểm tra xem đỉnh  $x$  có trong trạng thái  $S$  chưa, ta dùng phép  $\&$  và  $\ll$  như sau:  $s \& (1 \ll x)$

## Quy hoạch động trạng thái với bitmask

```
dp[0][1] = 1;
for (int s = 1; s < 1 << cityNum; s++) {
    // tập s không chứa thành phố đầu tiên thì bỏ qua
    if ((s & (1 << 0)) == 0) continue;
    // nếu đến thành phố n-1 mà chưa thăm đủ hết các thành phố thì bỏ qua
    if ((s & (1 << (cityNum - 1))) && s != ((1 << cityNum) - 1)) continue;
    for (int i = 0; i < cityNum; i++) {
        if ((s & (1 << end)) == 0) continue;
        // xét các tập s' (prev) không chứa đỉnh i
        int prev = s - (1 << i);
        for (int j : adj[i]) {
            if ((s & (1 << j))) {
                dp[s][i] += dp[prev][j];
                dp[s][i] %= MOD;
            }
        }
    }
}
cout << dp[(1 << city_num) - 1][city_num - 1] << '\n';
```

## Quy hoạch động trạng thái với bitmask

- Đặc điểm nhận dạng bài toán bitmask DP:
  - $n$  nhỏ ( $\sim 20$ ) vì  $2^n \sim 10^6$
  - có thể lưu được trạng thái dưới dạng dãy bit (chọn hoặc không chọn / đã đánh dấu hoặc chưa đánh dấu / đã thăm hoặc chưa thăm).
  - tại mỗi vị trí cần tính giá trị quy hoạch động, cần biết trạng thái hiện tại / trạng thái trước đó.

## Quy hoạch động trạng thái với bitmask

- Bài tập áp dụng:
  - [https://atcoder.jp/contests/dp/tasks/dp\\_o?lang=en](https://atcoder.jp/contests/dp/tasks/dp_o?lang=en)
- Tóm tắt:
  - $n$  bạn nam,  $n$  bạn nữ ( $1 \leq n \leq 21$ )
  - $a[i][j]=1$  nếu bạn nam thứ  $i$  phù hợp với bạn nữ thứ  $j$ , ngược lại  $a[i][j]=0$
  - **Yêu cầu:** Hãy đếm số cách có thể kết hợp thành  $n$  cặp đôi. Mỗi người chỉ có thể bắt cặp đúng với 1 người khác giới.



## Quy hoạch động trạng thái với bitmask

- Bài tập áp dụng:
  - <https://codeforces.com/contest/1316/problem/E>
- Tóm tắt:
  - Có  $n$  người ( $n \leq 10^5$ ,  $p \leq 7$ )
  - Chọn ra  $p$  cầu thủ tương ứng với  $p$  vị trí, người thứ  $i$  nếu chơi ở vị trí  $j$  sẽ có sức mạnh là  $s[i][j]$ .
  - Chọn thêm  $k$  người cổ vũ, người thứ  $i$  nếu được chọn sẽ có sức mạnh  $a[i]$ .
  - **Yêu cầu:** tổng sức mạnh lớn nhất.

# Quy hoạch động trạng thái với bitmask

- Bài tập áp dụng:
  - <https://usaco.org/index.php?page=viewproblem2&cpid=494>
- Tóm tắt:
  - Có  $n$  con bò ( $2 \leq n \leq 20$ )
  - Chiều cao, cân nặng, sức mạnh của con bò thứ  $i$  là  $h[i]$ ,  $w[i]$ ,  $s[i]$ .
  - **Yêu cầu:** Đặt các con bò chồng lên nhau sao cho đạt được tối thiểu chiều cao  $H$  với lượng cân nặng còn có thể đặt lên trên cùng là lớn nhất. ( $H \leq 10^9$ )
  - Điều kiện:  $s[i] \geq \sum w[j]$  với  $j$  là những con bò bên trên con bò thứ  $i$ .

## Quy hoạch động trạng thái với bitmask

- Bài tập áp dụng:
  - <https://usaco.org/index.php?page=viewproblem2&cpid=515>
- Tóm tắt:
  - Có  $n$  tập phim ( $1 \leq n \leq 20$ ), mỗi tập phim có thời lượng và các khoảng thời gian trình chiếu trong ngày.
  - **Yêu cầu:** Tìm cách để xem phim xuyên suốt từ thời điểm 0 đến thời điểm  $L$  với số lượng tập phim là ít nhất.
  - Điều kiện: có thể chuyển giữa các tập phim bất kỳ lúc nào, nhưng không được xem tập phim nào 2 lần.

## Quy hoạch động trạng thái với bitmask

- Bài tập áp dụng (khó):
  - <https://usaco.org/index.php?page=viewproblem2&cpid=1089>
  - <https://cses.fi/problemset/task/1653>
  - [https://oj.uz/problem/view/IZhO14\\_bank](https://oj.uz/problem/view/IZhO14_bank)
  - <https://open.kattis.com/problems/catandmice>